ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΜΑΚΕΔΟΝΙΑΣ
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Space Shooter

Made with Unity

Evgenia Dretaki
ΟΝΟΜΑ: Ευγενία Δρετάκη
ΑΕΜ: 4757

Πίνακας περιεχομένων

1.Elevator Pitch 3
2. Πληροφορίες Παιχνιδιού 3
3. Gameplay Loop 4
4. Κεντρικός Πυρήνας Μηχανισμών (Core Hook) 5
5. Πρόοδος & Επαναληψιμότητα 7
Κύματα (Waves) 7
Σκορ 7
Τέλος Παιχνιδιού 7
Ροή Power-Up 7
6. Τεχνικό Υπόβαθρο 7
Player Controls (Done_PlayerController.) 7
Auto-Fire & Triple Shot (Done_WeaponController) 8
Wave Spawning (Done_GameController) 8
AI Εχθρών 8
VFX & Cleanup 9
7. Οπτική Ταυτότητα & Αίσθηση 9
8. Πηγές 10

1. Elevator Pitch

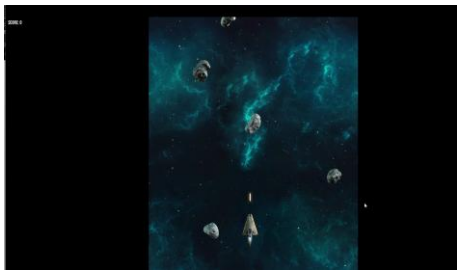
Retro 2D space shooter όπου ένα διαστημόπλοιο αποφεύγει εξωγήινους και περιστρεφόμενους αστεροειδείς. Ο παίκτης προσπαθεί να αυξήσει το score του χρησιμοποιώντας laser δημιουργώντας εκρήξεις.
Είναι χτισμένο στο Unity για ατελείωτη επαναληψιμότητα.

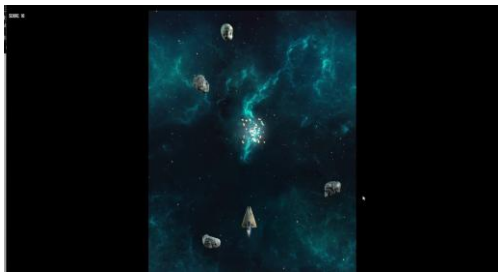
2. Πληροφορίες Παιχνιδιού

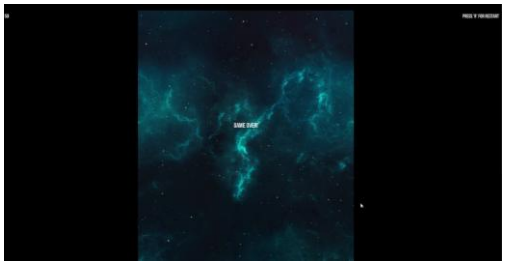
Είδος	Arcade Top-Down Shooter
Πλατφόρμα	PC (Unity) → Mobile/Web
Κοινό	Retro fans, ηλικίες 10+
Διάρκεια Session	2–5 λεπτά
Ομάδα	Solo Developer
Εργαλεία	Unity 202x, δωρεάν και ελαφριά assets

3. Gameplay Loop

• Εκκίνηση → Κίνηση & Shooting (mouse/keyboard)
→ Δημιουργία κύματος → Αποφυγή → Συλλογή Power-Up
→ Σκοράρισμα → Σύγκρουση → Restart με 'R'
→ Νέο High Score







Εικόνα 1: Shooting εναν αστεροειδή
Εικόνα 2: Η έκρηξη του αστεροειδή και update στα scores
Εικόνα 3: Έκρηξη του παίκτη -> Game Over
Εικόνα 4: Ο πάικτης πατάει "R" για να γίνει το Restart

4. Κεντρικός Πυρήνας Μηχανισμών (Core Hook)

4.1 Παίκτης

Διαστημόπλοιο

- Χρήση Rigidbody με ώθηση (speed 5–10)
- Περιστροφή/κλίση ανάλογα με την ταχύτητα (tilt rotation)
- Όρια κίνησης (xMin/Max, zMin/Max)
- Έλεγχος: WASD/βέλη για κίνηση + Fire (Mouse)
- Shootinga: lasers με ρυθμό 0.25s

4.2 Απειλές (Hazards)

Εχθροί: 1 τύπος εξωγήινου σκάφους (μωβ)

- Ελιγμοί ζιγκ-ζαγκ μέσω Done_EvasiveManeuver
- Dodge εύρους ±2–4

Αστεροειδείς: 3 παραλλαγές

- Τυχαία περιστροφή μέσω Done_RandomRotator

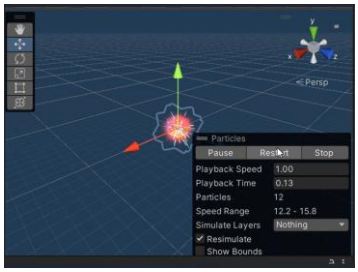
Waves:

- 5–10 hazards ανά κύμα, τυχαία διάταξη.

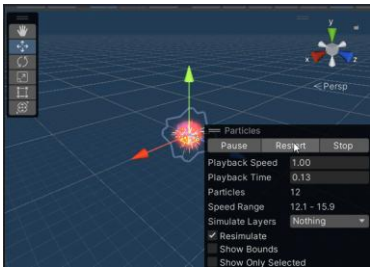
4.3 Συγκρούσεις

- OnTriggerEnter → Καταστροφή object

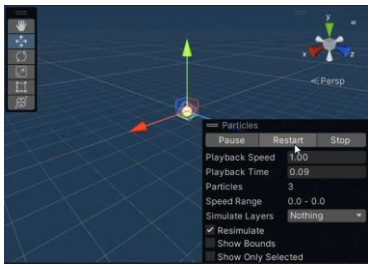
Καταστροφή Αστεροειδή

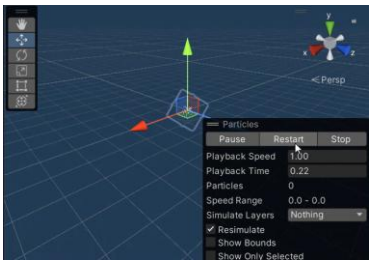


Καταστροφή Εχθρού



• Αν χτυπηθεί ο παίκτης → Άμεσο GameOver + playerExplosion





4.4 Πίνακας: Core Features

Feature	Script	Χρήση
Κίνηση & Shooting	Done_PlayerController	Fluid ανταπόκριση + auto-fire
Κύματα Εχθρών	Done_GameController	Άπειρη κλιμάκωση με coroutine
Ελιγμοί Εχθρών	Done_EvasiveManeuver	Ζιγκ-ζαγκ με smoothing

5. Πρόοδος & Επαναληψιμότητα

5.1 Κύματα (Waves)

- Άπειρα κύματα με σταδιακή αύξηση πυκνότητας
- hazardCount++ ή με tuning σε spawn rate

5.2 Σκορ

- +10 έως +50 ανά εχθρό
- GUIText HUD (upper-left)

5.3 Τέλος Παιχνιδιού

- 1 ζωή
- «Game Over!» + πάτημα R για restart
- High-score με PlayerPrefs

5.4 Ροή Power-Up

- 10% πιθανότητα να πέσει από εχθρό
- Ενεργοποίηση → Coroutine timer 10s → Επιστροφή σε single-shot

6. Τεχνικό Υπόβαθρο

Χτισμένο αποκλειστικά με Unity 2D physics.
Σύνολο scripts: 9
Ρυθμός καρέ: 60FPS

6.1 Player Controls (Done_PlayerController.)

Κάνει:

- Εισαγωγή input
- Κίνηση Rigidbody
- Περιορισμό ορίων
- Κλίση (tilt)
- Πυροβολισμούς

```
void Update ()
{
    if (Input.GetButton("Fire1") && Time.time > nextfire)
    {
        nextFire = Time.time + fireRate;
        Instantiate(shot, shotSpawn.position, shotSpawn.rotation);
        GetComponent<AudioSource>().Play ();
    }
}

void FixedUpdate ()
{
    float moveHorizontal = Input.GetAxis ("Horizontal");
    float moveVertical = Input.GetAxis ("Vertical");

    Vector3 movement = new Vector3 (moveHorizontal, 0.0f, moveVertical);
    GetComponent<Rigidbody>().velocity = movement * speed;

    GetComponent<Rigidbody>().position = new Vector3
    {
        Mathf.Clamp (GetComponent<Rigidbody>().position.x, boundary.xMin, boundary.xMax),
        0.0f,
        Mathf.Clamp (GetComponent<Rigidbody>().position.z, boundary.zMin, boundary.zMax)
    };

    GetComponent<Rigidbody>().rotation = Quaternion.Euler (0.0f, 0.0f, GetComponent<Rigidbody>().velocity.x * -tilt);
}
```

6.2 Auto-Fire & Triple Shot (Done_WeaponController)

- Βασικό shooting: InvokeRepeating
- Triple-Shot: bool flag + χρονόμετρο

```
void Start ()
{
    InvokeRepeating ("Fire", delay, fireRate);
}

void Fire ()
{
    Instantiate(shot, shotSpawn.position, shotSpawn.rotation);
    GetComponent<AudioSource>().Play();
}
```

6.3 Wave Spawning (Done_GameController)

Το κύμα γεννιέται με coroutine loop:

```
IEnumerator SpawnWaves ()
{
    yield return new WaitForSeconds (startWait);
    while (true)
    {
        for (int i = 0; i < hazardCount; i++)
        {
            GameObject hazard = hazards [Random.Range (0, hazards.Length)];
            Vector3 spawnPosition = new Vector3 (Random.Range (-spawnValues.x, spawnValues.x), spawnValues.y, spawnValues.z);
            Quaternion spawnRotation = Quaternion.identity;
            Instantiate (hazard, spawnPosition, spawnRotation);
            yield return new WaitForSeconds (spawnWait);
        }
        yield return new WaitForSeconds (waveWait);

        if (gameOver)
        {
            restartText.text = "Press 'R' for Restart";
            restart = true;
            break;
        }
    }
}
```

6.4 AI Εχθρών

- **Done_EvasiveManeuver:** Τυχαίοι στόχοι dodge
- **Done_RandomRotator:** Αστεροειδείς με περιστροφή
- **Done_Mover:** Βασική ταχύτητα κίνησης

```
IEnumerator Evade ()
{
    yield return new WaitForSeconds (Random.Range (startWait.x, startWait.y));
    while (true)
    {
        targetManeuver = Random.Range (1, dodge) * -Mathf.Sign (transform.position.x);
        yield return new WaitForSeconds (Random.Range (maneuverTime.x, maneuverTime.y));
        targetManeuver = 0;
        yield return new WaitForSeconds (Random.Range (maneuverWait.x, maneuverWait.y));
    }
}

void FixedUpdate ()
{
    float newManeuver = Mathf.MoveTowards (GetComponent<Rigidbody>().velocity.x, targetManeuver, smoothing * Time.deltaTime);
    GetComponent<Rigidbody>().velocity = new Vector3 (newManeuver, 0.0f, currentSpeed);
    GetComponent<Rigidbody>().position = new Vector3
    (
        Mathf.Clamp(GetComponent<Rigidbody>().position.x, boundary.xMin, boundary.xMax),
        0.0f,
        Mathf.Clamp(GetComponent<Rigidbody>().position.z, boundary.zMin, boundary.zMax)
    );

    GetComponent<Rigidbody>().rotation = Quaternion.Euler (0, 0, GetComponent<Rigidbody>().velocity.x * -tilt);
}
```

```
public class Done_RandomRotator : MonoBehaviour
{
    public float tumble;

    void Start ()
    {
        GetComponent<Rigidbody>().angularVelocity = Random.insideUnitSphere * tumble;
    }
}
```

```
public class Done_Mover : MonoBehaviour
{
    public float speed;

    void Start ()
    {
        GetComponent<Rigidbody>().velocity = transform.forward * speed;
    }
}
```

6.5 VFX & Cleanup

• **Done_DestroyByContact**: Εκρήξεις και Score

```
void OnTriggerEnter (Collider other)
{
    if (other.tag == "Boundary" || other.tag == "Enemy")
    {
        return;
    }

    if (explosion != null)
    {
        Instantiate(explosion, transform.position, transform.rotation);
    }

    if (other.tag == "Player")
    {
        Instantiate(playerExplosion, other.transform.position, other.transform.rotation);
        gameController.GameOver();
    }

    gameController.AddScore(scoreValue);
    Destroy (other.gameObject);
    Destroy (gameObject);
}
```

• **Done_DestroyByTime**: Διαγραφή bullets/particles

```
public class Done_DestroyByTime : MonoBehaviour
{
    public float lifetime;

    void Start ()
    {
        Destroy (gameObject, lifetime);
    }
}
```

• **Done_BGScroller**: Άπειρο scrolling φόντο

```
void Start ()
{
    startPosition = transform.position;
}

void Update ()
{
    float newPosition = Mathf.Repeat(Time.time * scrollSpeed, tileSizeZ);
    transform.position = startPosition + Vector3.forward * newPosition;
}
```

7. Οπτική Ταυτότητα & Αίσθηση

7.1 Στυλ

• Pixel-style sprites
• Neon lasers (cyan/orange)
• Particles: jet engines, sparks, shockwaves
• Φόντο: Nebula tiles + “junkyard space”

7.2 Ήχος

• Pew lasers
• Boom εκρήξεις
• Απόδοση: λιτό και retro

8. Πηγές

Spaceships

1. Kenney – “Spaceshooter Redux”
   https://kenney.nl/assets/spaceshooter-redux
2. OpenGameArt – “Top-Down Space Ships Pack”
   https://opengameart.org/content/space-ships-top-down

Enemies

3. Kenney – “Space Shooter Extension Pack”
   https://kenney.nl/assets/space-shooter-extension

Αστεροειδείς

4. Kenney – “Asteroids Pack”
   https://kenney.nl/assets/asteroids

Power-Ups

5. Kenney – “Power-Ups”
   https://kenney.nl/assets/power-ups

Εκρήξεις / VFX

6. Kenney – “Explosion Pack”
   https://kenney.nl/assets/explosion
7. OpenGameArt – “Explosion Sheet”
   https://opengameart.org/content/explosion-sheet

Lasers / Bullets

8. Kenney – “Laser Pack”
   https://kenney.nl/assets/laser-bullet

Background

9. Kenney – “Space Shooter Assets”
   https://kenney.nl/assets/space-shooter

Audio

10. OpenGameArt – “Laser Fire SFX”
    https://opengameart.org/content/laser-fire
11. OpenGameArt – “Explosion SFX”
    https://opengameart.org/content/explosion-0